Practice Lab 3 Worksheet

Ques1. Implement Bubble Sort, Selection Sort, Linear Search and Binary Search on your systems and perform the empirical analysis for all these algorithms.

Bubble Sort

n	T(n)	T(n)/n	$T(n)/n^2$	$T(n)/n^3$
10				
50				
100				
1000				
10,000				
50,000				
1,00,000				
10,00,000				

Selection Sort

n	T(n)	T(n)/n	$T(n)/n^2$	$T(n)/n^3$
10				
50				
100				
1000				
10,000				
50,000				
1,00,000				
10,00,000				

Linear Search

n	T(n)	T(n)/(n/2)	T(n)/n	$T(n)/n^2$
10				
50				
100				
1000				
10,000				
50,000				
1,00,000				
10,00,000				

Binary Search

n	T(n)	T(n)/(log (n/2))	T(n)/(log n)	T(n)/(n/2)
10				
50				
100				
1000				
10,000				
50,000				
1,00,000				
10,00,000				

Ques2. What is the time complexity of the following program?

(a)
```
int fun(int n)
{
        int count = 0;
        for (int i = n; i > 0; i /= 2)
                for (int j = 0; j < i; j++)
                        count += 1;
        return count;
}
```

(b)
```
for (int j = 1 to n) {
  int k = j
  while (k < n) {
        sum += a[k]*b[k]
        k ++;
  }
  }
```

(c)
```
int fun(int n)
{
int count = 0;
for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
                for(k = 0; k < n; k++)
                        count += 1;
return count;
}
```

(d)
```
int a = 0, b = 0;
for (i = 0; i < N; i++) {
        a = a + rand();
}
for (j = 0; j < M; j++) {
        b = b + rand();
}
```

(e)
```
int a = 0;
for (i = 0; i < N; i++) {
        for (j = N; j > i; j--) {
                a = a + i + j;
        }
}
```

(f)
```
int i, j, k = 0;
for (i = n / 2; i <= n; i++)
{
        for (j = 2; j <= n; j = j * 2)
        {
                k = k + n / 2;
        }
}
```

(g)
```
int a = 0, i = N;
while (i > 0)
{
        a += i;
        i /= 2;
}
```

(h)
```
int fun(int n)
{
        for (int i = 1; i <= n; i++)
        {
                for (int j = 1; j < n; j += i)
                {
                        // Some O(1) task
                }
        }
}
```

Ques3. For the functions, n^k and c^n, what is the asymptotic relationship between these functions? Assume that k >= 1 and c > 1 are constants.

Ques4. Decide whether these statements are True or False:

1. If f(n) = Θ(g(n)) and g(n) = Θ(h(n)), then h(n) = Θ(f(n))
2. If f(n) = O(g(n)) and g(n) = O(h(n)), then h(n) = Ω(f(n))
3. If f(n) = O(g(n)) and g(n) = O(f(n)) then f(n) = g(n)
4. $\frac{n}{100}$ = Ω(n)

How to Determine Complexities

In general, how can you determine the running time of a piece of code? The answer is that it depends on what kinds of statements are used.

1. Sequence of statements

```
statement 1;
statement 2;
  ...
statement k;
```

   (Note: this is code that really is exactly k statements; this is **not** an unrolled loop like the N calls to *add* shown above.) The total time is found by adding the times for all statements:

   total time = time(statement 1) + time(statement 2) + ... + time(statement k)

   If each statement is "simple" (only involves basic operations) then the time for each statement is constant and the total time is also constant: O(1). In the following examples, assume the statements are simple unless noted otherwise.

2. if-then-else statements

```
if (condition) {
   sequence of statements 1
}
else {
   sequence of statements 2
}
```

   Here, either sequence 1 will execute, or sequence 2 will execute. Therefore, the worst-case time is the slowest of the two possibilities: max(time(sequence 1), time(sequence 2)). For example, if sequence 1 is O(N) and sequence 2 is O(1) the worst-case time for the whole if-then-else statement would be O(N).

3. for loops

```
for (i = 0; i < N; i++) {
   sequence of statements
}
```

   The loop executes N times, so the sequence of statements also executes N times. Since we assume the statements are O(1), the total time for the for loop is N * O(1), which is O(N) overall.

4. Nested loops

   First we'll consider loops where the number of iterations of the inner loop is independent of the value of the outer loop's index. For example:

```
for (i = 0; i < N; i++) {
   for (j = 0; j < M; j++) {
      sequence of statements
   }
}
```

The outer loop executes N times. Every time the outer loop executes, the inner loop executes M times. As a result, the statements in the inner loop execute a total of N * M times. Thus, the complexity is O(N * M). In a common special case where the stopping condition of the inner loop is j < N instead of j < M (i.e., the inner loop also executes N times), the total complexity for the two loops is $O(N^2)$.

Now let's consider nested loops where the number of iterations of the inner loop depends on the value of the outer loop's index. For example:

```
for (i = 0; i < N; i++) {
   for (j = i+1; j < N; j++) {
      sequence of statements
   }
}
```

Now we can't just multiply the number of iterations of the outer loop times the number of iterations of the inner loop, because the inner loop has a different number of iterations each time. So let's think about how many iterations that inner loop has. That information is given in the following table:

Value of i	Number of iterations of inner loop
0	N
1	N-1
2	N-2
...	...
N-2	2
N-1	1

So we can see that the total number of times the sequence of statements executes is: N + N-1 + N-2 + ... + 3 + 2 + 1. We've seen that formula before: the total is $O(N^2)$.